# SwiftDrop Parcel Delivery & Courier Management System

## Team Work Division, Responsibilities, Deliverables & Collaboration Plan

*Version 1.0 | Proposed team baseline | 21 October 2026*

# 1. Purpose and how to use this document

This document turns the selected capability-based division (Option 1) into an implementation agreement for a 10-member team. It defines each member's mission, responsibilities, deliverables, required inputs, dependencies, and acceptance criteria. Treat it as a proposed team baseline: decisions marked as needing agreement must be confirmed by the team before implementation.

The allocation gives each member a primary ownership area, but it does not mean that work is isolated. Backend contracts, database changes, authorization, event delivery, frontend integration, deployment, and testing require collaboration. All members own the quality of their own work.

# 2. Architecture baseline assumed by the division

- Core API: a Go modular monolith containing Identity & Access, Merchant Integration, Admin-facing business capabilities, Proof & Delivery Records, Dispatch & Couriers, Pricing, and Parcel Management.
- Tracking API: a separate Go service/deployment for public and authenticated tracking, designed to scale independently and read current authoritative parcel/status data.
- Async integration: PostgreSQL transactional outbox → Outbox Relay → RabbitMQ → Event Worker for notifications and merchant webhooks.
- Frontend: React + TypeScript + Vite + Tailwind CSS, with customer/merchant journeys and dispatcher/admin journeys owned separately.
- Platform: Docker Compose for local development; Kubernetes resources, configuration, health probes, scaling, and observability for the deployment bonus.
- AI assistant: separate Python/FastAPI service with limited read-only tools that call backend APIs; MCP is an implementation option, not a reason to delay core requirements.
- PostgreSQL is the authoritative source for parcel state. RabbitMQ transports events; Redis may be used for cross-replica live-update fan-out if the team chooses it, not as the source of truth.

## Member 1 — Identity, Access & Security

Build the shared identity and authorization foundation used by every protected workflow.

**Primary technologies:** Go; selected HTTP framework; PostgreSQL; JWT/session library; Google OAuth 2.0 / OpenID Connect integration.

### Responsibilities

- Define and implement user registration, login, logout/session handling, and account profile basics.
- Implement Google sign-in using the agreed OIDC flow; validate issuer, audience, signature, expiry, nonce/state, and callback handling.
- Implement authentication middleware and role-based access control for customer, merchant, courier, dispatcher, and admin roles.
- Provide reusable authorization helpers for ownership and tenant checks; document what each role may do.
- Implement admin user/role-management endpoints if not assigned to the Admin UI owner; coordinate endpoint ownership with Member 7.
- Apply password hashing where passwords are used, secure cookie/token handling as agreed, rate limiting for sensitive endpoints, and safe error responses.
- Write unit and integration tests for login, token validation, role checks, and forbidden cross-user access.

### Deliverables

- Authentication and authorization module with documented configuration.
- Versioned API contract for auth/profile endpoints and role/permission matrix.
- Google OIDC setup guide with required redirect URI and environment variables (no secrets committed).
- Reusable middleware/helpers and example usage for other backend modules.
- Automated tests covering success, invalid credentials/token, expiry, missing role, and object-ownership denial.
- README section describing local setup, seed/test users, and security assumptions.

### Inputs required

- Approved role definitions and user journeys from the SRS.
- Chosen token/session strategy, frontend callback URLs, Google OAuth client configuration.
- Shared user and role schema, API error format, and configuration conventions.

### Dependencies / coordination

- Must publish the auth contract and middleware interface early so Members 2–5 and 8 can integrate.
- Works with Members 6–7 on frontend login flows and role-specific navigation.
- Every module owner remains responsible for enforcing authorization on their own endpoints.

### Acceptance criteria

- Protected endpoints reject missing, invalid, expired, or insufficiently privileged credentials.
- Users cannot read or mutate another customer's parcel or another merchant's data by changing an ID.
- Google login works in the agreed local/demo environment, or documented mock/development fallback is provided if credentials are unavailable.
- Tests run in CI/local instructions and no credentials are committed.

## Member 2 — Parcel Management & Pricing

Own parcel creation and the core parcel business data, including consistent fee estimation and lifecycle records.

**Primary technologies:** Go; PostgreSQL; REST/JSON; shared validation and error conventions.

### Responsibilities

- Implement customer parcel booking with pickup/drop-off details, parcel size/weight, recipient/contact fields, and input validation.
- Implement pricing/fee estimation using a documented, configurable baseline formula agreed by the team.
- Persist parcel records, tracking number, initial status, pricing snapshot, and status-history records as appropriate.
- Implement parcel history and receipt data endpoints; coordinate UI presentation with Member 6.
- Define and enforce legal parcel-state transitions with Member 3 and Member 5; reject invalid or duplicate transitions.
- Support idempotent booking requests so client retries do not accidentally create duplicate parcels.
- Create domain events/outbox records through the agreed shared event interface, in coordination with Member 8.
- Write tests for validation, fee calculations, idempotency, history, and invalid state transitions.

### Deliverables

- Parcel booking, fee estimate, history, and receipt-data endpoints with OpenAPI examples.
- Pricing formula/rules document with example calculations, units, currency, and edge cases.
- Database migrations and indexes for parcel, tracking number, price snapshot, and relevant history data.
- Tests for valid/invalid bookings, boundary values, repeated idempotency key, and pricing examples.
- Seed data/demo scenario and module README.

### Inputs required

- Approved parcel fields, status lifecycle, currency, pricing formula, and validation rules.
- Database conventions and shared API response/error contract.
- Auth middleware and authorization rules from Member 1; outbox interface from Member 8.

### Dependencies / coordination

- Coordinate parcel schema and lifecycle with Members 3, 5, and 8 before migrations become fixed.
- Coordinate customer-facing endpoint requirements with Member 6.
- Tracking API must read authoritative parcel/status data without relying on a stale projection.

### Acceptance criteria

- A valid booking creates one parcel with a unique public tracking number and a reproducible fee snapshot.
- Invalid required fields and unsupported sizes/weights receive structured validation errors.
- Repeated booking with the same idempotency key does not create duplicates.
- History and receipt data are available only to authorized owners/roles, except deliberately public tracking fields.

## Member 3 — Dispatch, Couriers & Proof of Delivery

Implement operational assignment and courier execution, including auditable delivery attempts and proof.

**Primary technologies:** Go; PostgreSQL; REST/JSON; object-storage abstraction or agreed local demo storage for proof files.

### Responsibilities

- Implement dispatcher views/endpoints for unassigned parcels, courier assignments, and workload counts.
- Implement assignment/reassignment rules and protect against conflicting concurrent assignments.
- Implement courier views for assigned deliveries and permitted parcel details.
- Implement courier status updates with notes, proof metadata/photo reference, and delivery failure reason.
- Represent delivery attempts separately where needed so a failed attempt does not silently erase history or create an invalid state transition.
- Validate that a courier can update only deliveries assigned to them, subject to the agreed dispatcher override rules.
- Coordinate status transition definitions with Members 2 and 5 and publish events through Member 8's shared interface.
- Write tests for assignment conflicts, unauthorized courier updates, successful delivery, failed attempt, and proof validation.

### Deliverables

- Dispatch and courier REST endpoints with request/response examples.
- Assignment and delivery-attempt schema/migrations, including audit timestamps and actor IDs.
- Documented status/attempt rules and allowed dispatcher overrides.
- Proof-of-delivery upload/reference workflow suitable for the demo, with file type/size validation if uploads are supported.
- Automated authorization, concurrency, and workflow tests.
- Dispatcher/courier demo data and module README.

### Inputs required

- Approved parcel lifecycle and failed-delivery semantics.
- Courier/dispatcher role rules and authorization helpers from Member 1.
- Parcel schema from Member 2 and event/outbox contract from Member 8.
- UI requirements from Member 7.

### Dependencies / coordination

- Must agree with Member 2 on the authoritative parcel/status tables and avoid duplicate competing definitions.
- Member 7 consumes dispatch/admin endpoints; Member 5 consumes current status and history.
- Member 8 must receive status/assignment events via the shared event contract.

### Acceptance criteria

- Dispatchers can assign eligible parcels and see workload counts.
- Couriers can only act on their own assigned deliveries unless an explicit override is authorized.
- Each status update is auditable with actor, timestamp, and relevant note/proof/failure reason.
- Concurrent assignments do not silently produce two active courier assignments for one parcel.

## Member 4 — Merchant Integration

Provide secure merchant onboarding and system-to-system parcel creation/status integration.

**Primary technologies:** Go; PostgreSQL; REST/JSON; API keys or OAuth 2.0 client credentials (one strategy selected for the baseline).

### Responsibilities

- Implement merchant registration/application and approval-state handling in coordination with Member 7's admin workflow.
- Implement merchant API credential issuance, rotation/revocation, and secure credential storage/verification.
- Implement tenant-scoped merchant endpoints for creating and viewing the merchant's own parcels.
- Implement webhook configuration endpoints and validation of destination URLs, events, and enabled/disabled state.
- Enforce tenant isolation at every query and mutation; never trust merchant_id supplied by the caller without checking credentials.
- Document merchant API authentication, idempotency, request limits, error responses, and versioning.
- Coordinate delivery status event subscriptions with Member 8, which owns outbound webhook delivery/retries.
- Write tests for valid/invalid credentials, cross-tenant access, duplicate requests, and webhook configuration validation.

### Deliverables

- Merchant onboarding/approval and API credential endpoints.
- Merchant API specification and sample requests (curl/Postman collection).
- Tenant-isolation tests and credential rotation/revocation tests.
- Webhook configuration schema/API; clear boundary that Member 4 configures webhooks while Member 8 delivers them.
- Merchant integration quick-start and demo merchant.

### Inputs required

- Merchant approval rules and role matrix.
- Shared parcel creation contract from Member 2 and auth/security conventions from Member 1.
- Event names/payloads and webhook retry contract from Member 8.
- Merchant dashboard needs from Member 6.

### Dependencies / coordination

- Member 7 owns the admin UI for merchant approval; Member 4 owns the backend capability and API contract.
- Member 8 owns delivery attempts, retries, signatures, and DLQ for outbound webhooks.
- Member 6 uses the merchant endpoints for the dashboard.

### Acceptance criteria

- A merchant can access only its own records and create parcels through documented authenticated endpoints.
- Revoked credentials stop working; credentials are not stored or logged in plaintext.
- Webhook configuration is validated and cannot be used to target prohibited internal/private network destinations.
- Outbound webhook retry behavior is demonstrable through Member 8's worker.

## Member 5 — Tracking API & Real-Time Updates

Provide a separately deployable, independently scalable tracking read path with live status updates.

**Primary technologies:** Go; REST/JSON; Server-Sent Events (SSE) or the team-approved realtime mechanism; PostgreSQL read-only access; optional Redis Pub/Sub fan-out.

### Responsibilities

- Build the separately deployed Tracking API for public tracking by tracking number and authenticated parcel tracking/history.
- Expose only the deliberately approved public fields; do not reveal full addresses, phone numbers, merchant secrets, or other private information.
- Read current authoritative parcel/status data from PostgreSQL (or an explicitly approved consistent read interface); do not create an unapproved stale tracking projection.
- Implement SSE updates if feasible; define reconnect behavior, heartbeat, event IDs, and client fallback/polling behavior.
- If multiple Tracking API replicas must deliver the same live updates, coordinate Redis Pub/Sub or another fan-out mechanism; PostgreSQL remains the source of truth.
- Add caching only if invalidation and freshness behavior are explicit; do not let cache become authoritative.
- Implement rate limiting or coordinate it with the gateway, plus tests for privacy, unknown tracking IDs, and realtime reconnect.

### Deliverables

- Tracking API service, Dockerfile, deployment config, and API/SSE contract.
- Public/private tracking field matrix and threat/privacy notes.
- Realtime event schema and frontend integration example.
- Tests for public data minimization, unknown IDs, authorization, event delivery, reconnect/fallback, and replica fan-out if implemented.
- Load-test notes and module README.

### Inputs required

- Authoritative parcel/status schema from Members 2 and 3.
- Approved public tracking disclosure policy.
- Gateway routing/auth rules and frontend expectations.
- Event/status update mechanism and optional Redis configuration.

### Dependencies / coordination

- Member 2/3 status changes must commit to PostgreSQL before tracking can show them as current.
- Member 8 can publish domain events for fan-out if the design is agreed; otherwise define another explicit mechanism.
- Member 6 consumes the tracking contract; Member 9 deploys/scales the service.

### Acceptance criteria

- Tracking number returns the latest committed status and chronological history.
- Public responses omit sensitive customer and delivery details.
- Live update reaches connected clients within the agreed target under demo conditions; reconnect or polling fallback works.
- Tracking API can be scaled independently from Core API without starting unrelated Core API replicas.

## Member 6 — Customer & Merchant Frontend

Deliver the customer and merchant-facing web journeys against published backend contracts.

**Primary technologies:** React; TypeScript; Vite; Tailwind CSS; agreed form/state/query libraries if any.

### Responsibilities

- Implement customer registration/login and Google sign-in UI based on Member 1's flow.
- Implement customer parcel booking form, live fee estimate, validation display, booking confirmation, receipt view, and parcel history.
- Implement tracking screen with current status/history and SSE or polling fallback.
- Implement merchant dashboard for creating/viewing merchant parcels and configuring/seeing integration status as agreed.
- Implement loading, empty, success, and error states; accessible labels and responsive layouts.
- Use typed API clients and environment-based base URLs; do not hardcode secrets or assume API success.
- Create component and workflow tests for the critical customer and merchant journeys.

### Deliverables

- Customer and merchant UI routes/components, responsive styling, and shared UI conventions.
- Typed API client/service layer and environment example file.
- Customer booking/tracking/receipt flow and merchant parcel dashboard.
- Frontend tests or documented manual test checklist for key flows.
- Frontend Dockerfile/build instructions and README.

### Inputs required

- Auth, parcel, merchant, and tracking API contracts from Members 1, 2, 4, and 5.
- Design direction, required fields, validation rules, and error response format.
- Shared environment/configuration conventions.

### Dependencies / coordination

- Can begin with mocked API responses while contracts stabilize.
- Member 1 supports login/OIDC integration; Members 2/4/5 support API integration.
- Member 9 packages and deploys the frontend.

### Acceptance criteria

- Customer can register/sign in, estimate fee, book, view receipt/history, and track a parcel.
- Merchant can sign in and use only merchant-authorized screens/data.
- API failures are visible and understandable; sensitive values are not exposed in browser logs.
- Critical flows work in the agreed local Compose/demo environment.

## Member 7 — Dispatcher/Admin Frontend & API Gateway

Deliver operational/admin screens and configure a clear external API entry point.

**Primary technologies:** React; TypeScript; Tailwind CSS; Kong or another team-approved gateway; Docker/Kubernetes routing.

### Responsibilities

- Implement dispatcher dashboard for unassigned parcels, assignments, reassignment where allowed, and courier workload counts.
- Implement admin screens for user/merchant approval, zones, pricing-rule management, and basic reports within the agreed scope.

- Enforce UI-level role navigation for usability while relying on backend authorization as the real security boundary.
- Configure gateway routes for customer/merchant/admin/tracking APIs and define where authentication, rate limiting, CORS, and request-size limits belong.
- Keep gateway and Kubernetes Ingress responsibilities clear: Ingress exposes/routes to the gateway; the gateway applies API-level routing/policies. Avoid unnecessary proxy layers.
- Provide gateway configuration as version-controlled files and coordinate route changes with Members 1, 4, 5, 8, and 9.
- Write frontend tests/manual scenarios for dispatcher/admin workflows and smoke tests for gateway routes.

### Deliverables

- Dispatcher and admin UI screens with role-aware navigation.
- Gateway route/policy configuration and route map.
- Documented responsibility split between Ingress and gateway.
- Admin API contract needs and a list of endpoints consumed.
- Tests/checklist for approval, assignment, pricing/zone administration, and route behavior.
- Module README and demo script.

### Inputs required

- Admin/dispatcher API contracts from Members 1, 3, and 4.
- Pricing and zone model from Member 2 and team-approved business rules.
- Gateway decision, domain/port map, and deployment requirements from Member 9.

### Dependencies / coordination

- Member 3 owns dispatch backend logic; Member 4 owns merchant onboarding backend; Member 7 owns their user interface.
- Member 1 remains responsible for backend authorization.
- Member 9 owns cluster deployment integration and validates gateway/Ingress connectivity.

### Acceptance criteria

- Dispatcher/admin users can complete assigned operational workflows using backend APIs.
- Unauthorized roles do not see irrelevant navigation, and backend still rejects forbidden requests.
- Gateway routes to the correct service and applies agreed policies without exposing internal services unnecessarily.
- Gateway config can be reproduced from version control.

## Member 8 — Event-Driven Backend & Integrations

Own reliable asynchronous delivery of domain events, notifications, and merchant webhooks.

**Primary technologies:** Go; RabbitMQ; PostgreSQL transactional outbox; worker processes; email/SMS provider adapter or safe demo sink.

### Responsibilities

- Define event envelope conventions: event_id, event_type, schema_version, occurred_at, aggregate/parcel ID, correlation ID, and payload.
- Implement the transactional outbox integration contract so domain changes and outbox rows commit atomically in the same PostgreSQL transaction.
- Build a separately deployable Outbox Relay that claims eligible rows safely, publishes persistent messages, uses publisher confirms, and marks records published only after confirmation.
- Build Event Worker consumers for notification delivery and outbound merchant webhooks; use manual acknowledgements and idempotent processing.
- Implement bounded exponential retry with jitter, attempt limits, dead-letter queue (DLQ), operational visibility, and replay/runbook guidance.
- Sign webhook payloads, include timestamps/event IDs, document receiver verification, and protect outbound HTTP against SSRF and unsafe redirects/private destinations.
- Implement reconciliation command/job for stale outbox rows, stuck deliveries, and missing expected records; recover safely without bypassing the relay or inventing parcel state.
- Provide local fake email/webhook receivers so the demo does not depend on real paid providers.
- Write tests for publication, duplicate delivery, worker crashes/retries, DLQ, webhook signatures, and reconciliation.

### Deliverables

- Outbox table/interface, relay service, event worker service, queue definitions, and RabbitMQ configuration.
- Event catalog and versioned payload schemas/examples.
- Notification adapter and merchant webhook delivery pipeline with retries/DLQ.
- Reconciliation command and Kubernetes CronJob contract (Member 9 deploys it).
- Integration tests proving database commit → outbox → RabbitMQ → consumer → notification/webhook.
- Runbook for retrying/replaying failed deliveries safely; module README.

### Inputs required

- Domain event requirements from Members 2, 3, and 4.
- Shared database transaction conventions and event/outbox interface.
- RabbitMQ topology/configuration and environment conventions from Member 9.
- Webhook configuration data/API from Member 4.

### Dependencies / coordination

- Must agree on outbox schema and transaction boundary before other backend members finalize event-producing code.
- Member 9 deploys and monitors relay, worker, RabbitMQ, and reconciliation CronJob.
- Member 10 assists with cross-flow integration tests; each producer still owns its domain correctness.

### Acceptance criteria

- Committed business changes have a durable outbox event in the same transaction.
- Unpublished events are retried safely; publication is not marked successful before broker confirmation.
- Duplicate messages do not create duplicate externally visible effects.
- Failed deliveries reach DLQ after bounded retries and can be investigated/replayed safely.
- Integration tests demonstrate the full async flow and recovery path.

## Member 9 — DevOps, Kubernetes & Observability

Make the full system reproducible locally and deployable/observable on Kubernetes for the assignment bonus.

**Primary technologies:** Docker; Docker Compose; Kubernetes; Helm; Ingress controller; Prometheus/Grafana; optional KEDA.

### Responsibilities

- Create and maintain Dockerfiles for frontend, Core API, Tracking API, Outbox Relay, Event Worker, and AI Assistant; coordinate ownership with service developers.
- Create local Docker Compose for app services and local dependencies needed for development/demo; clearly separate local containers from any managed services.
- Create Kubernetes Deployments/Services for required workloads and database/broker according to the assignment's deployment scope.
- Create ConfigMaps and Secrets templates; commit examples/placeholders only, never actual credentials.
- Configure Ingress/controller and gateway connectivity with Member 7; define health/readiness/liveness probes, resource requests/limits, and graceful shutdown.
- Configure HPA for suitable stateless APIs and KEDA or an equivalent queue-depth scaler for the Event Worker if time allows and dependencies support it.
- Create Helm chart/templates and a values example if pursuing the Helm bonus.
- Add Prometheus scrape config and a practical Grafana dashboard for request rate/latency/errors, worker failures, queue depth, and outbox backlog where metrics exist.
- Write deployment troubleshooting and demo instructions; help all members reproduce their services.

### Deliverables

- Root Docker Compose file and per-service Dockerfiles/build instructions.
- Kubernetes manifests and/or Helm chart for required services, database, broker, gateway/Ingress, ConfigMaps, and Secrets references.
- Health checks, resource configuration, and scaling configuration.
- Prometheus/Grafana configuration and dashboard JSON if time allows.

- Deployment diagram, environment-variable inventory, setup/teardown commands, and troubleshooting guide.
- Verified clean-clone setup and deployment demonstration.

### Inputs required

- Service ports, startup commands, environment variables, health endpoints, and Docker build contexts from all service owners.
- Chosen database/broker deployment approach and secret names.
- Gateway route plan from Member 7 and scaling metrics from Members 5/8.

### Dependencies / coordination

- Each service owner must provide a working Dockerfile, health endpoint, config sample, and smoke test by the agreed integration milestone.
- Member 9 coordinates packaging/deployment but is not expected to debug all application logic alone.
- Full Kubernetes database/broker deployment should be included if required for the bonus rubric; if managed services are used, document the trade-off and verify rubric compatibility.

### Acceptance criteria

- A new team member can start the documented local stack from a clean clone.
- Required Kubernetes workloads become Ready and communicate through Services.
- Secrets are injected from Kubernetes Secrets/environment rather than committed files.
- Readiness/liveness probes and resource requests/limits are present for appropriate workloads.
- Scaling/deployment and monitoring demo is reproducible, with no claim that optional components are complete if they are not.

## Member 10 — AI Assistant/MCP & Cross-Workflow Quality

Implement a narrowly scoped AI delivery assistant and coordinate end-to-end verification without becoming the sole tester.

**Primary technologies:** Python; FastAPI; LLM provider SDK; MCP tools if feasible; pytest; API/E2E test tooling.

### Responsibilities

- Implement a separate AI Assistant API/service with controlled tools for tracking a parcel, retrieving authorized delivery history, and estimating a delivery fee.
- Keep tools read-only for the baseline; do not allow direct database access or arbitrary shell/network access.
- Validate tool inputs, apply timeouts and bounded retries, redact sensitive data, and return a safe fallback when the model/provider is unavailable.
- Preserve user authorization: public tracking can only return public fields; authenticated history must be checked against the caller's permissions.
- Use MCP for tool exposure if time and team skill allow; document tool schemas and permissions. The business APIs remain authoritative.
- Create a cross-workflow test plan and coordinate end-to-end tests for customer booking, tracking, courier delivery, merchant webhook, authorization, and failure/recovery.
- Track integration defects, verify fixes with owning members, and ensure each member writes tests for their own module.
- Help Member 8 verify event-flow and webhook failure cases if the AI feature is smaller than expected.

### Deliverables

- AI Assistant service, tool definitions/contracts, configuration example, and README.
- Tests for tool validation, authorization, prompt-injection boundaries, provider timeout/failure, and sensitive-data minimization.
- End-to-end test matrix, execution results, bug log, and final integration checklist.
- Test report linking requirements to test cases and recording known limitations.
- Demo script showing safe tracking/fee questions and at least one graceful failure case.

### Inputs required

- Tracking API and fee estimate contracts from Members 2 and 5.
- Auth/permission rules from Member 1.
- Team's selected LLM/provider and MCP scope; no credentials in source control.
- Acceptance criteria and environments from all members.

### Dependencies / coordination

- AI must call supported backend APIs/tools rather than querying databases directly.
- Members 1–9 remain responsible for their own unit/integration tests; Member 10 coordinates end-to-end coverage.
- Keep AI scope small until core parcel booking, tracking, dispatch, and event integration are stable.

### Acceptance criteria

- Assistant answers supported tracking/fee questions through explicit tools and does not invent parcel facts when tools fail.
- Unauthorized history access is denied and sensitive fields are not exposed.
- Core end-to-end test scenarios are repeatable and results/defects are recorded.
- AI is not allowed to block completion of the core delivery-management requirements.

## 4. Shared team rules and ownership boundaries

**Project coordinator / decision log:** Team elects one coordinator (not necessarily Member 9). Coordinator tracks milestones, decisions, risks, blockers, and meeting notes; does not own everyone else's implementation.

**API contracts:** Every backend owner publishes endpoint, auth requirement, request/response examples, validation rules, error codes, and version before frontend integration. OpenAPI is recommended.

**Database ownership:** Agree a single canonical PostgreSQL schema/migration strategy. Each feature owner owns migrations for their tables; one nominated database coordinator reviews naming, foreign keys, indexes, and migration ordering. Avoid duplicate parcel/status tables.

**Authorization:** Member 1 supplies reusable auth primitives, but every endpoint owner enforces role, object ownership, and merchant tenant isolation. Hiding a frontend button is not authorization.

**Events:** Member 8 publishes event envelope and outbox interfaces early. Producers must write outbox events in the same transaction as business state changes; consumers must be idempotent.

**Testing:** Each member writes and runs tests for their own module. Member 10 coordinates system tests and reporting; this is not a transfer of test responsibility.

**Documentation:** Each member maintains a concise module README, configuration list, API examples, tests, known limitations, and a short demo script.

**Secrets:** Commit .env.example with placeholders only. Never commit OAuth client secrets, database passwords, JWT signing keys, API keys, or webhook secrets.

**Branching/reviews:** Use short-lived feature branches and pull requests. At least one teammate reviews changes; migrations, auth, event, and deployment changes should receive extra review.

**Definition of done:** Code compiles, formatter/linter passes, tests pass, migration/setup instructions are updated, no secrets are committed, API contract matches implementation, and the feature can be demonstrated.

**Integration cadence:** Integrate continuously. Keep main branch runnable; use mocks for unimplemented dependencies; avoid waiting until all modules are finished.

**Demo readiness:** Seed deterministic demo users, merchants, couriers, parcels, and webhook receiver. Document demo credentials safely in local-only setup instructions.

## 5. Integration milestones

**M0 — Agree foundations (first):** Owners: All members. Confirm scope, roles, status machine, pricing formula/currency, failed-attempt behavior, auth strategy, gateway choice, event envelope, DB migration approach, and repo conventions.

**M1 — Contracts & skeletons:** Owners: Members 1–9; Member 10 drafts test plan. Publish API/event contracts, create service skeletons, config examples, health endpoints, initial schema/migrations, frontend route skeletons, Compose baseline.

**M2 — Core vertical slices:** Owners: Members 1–7. Working auth, customer booking/pricing, merchant onboarding/order API, dispatch/courier workflow, tracking API, customer/merchant/admin screens; mock unfinished integrations.

**M3 — Async integration:** Owners: Member 8 with producers 2–4; Member 9 supports broker. Transactional outbox, relay, RabbitMQ, notifications/webhooks, retries/DLQ, reconciliation and integration tests.

**M4 — Platform & AI:** Owners: Members 9–10; all service owners. Kubernetes manifests/Helm, probes/config/secrets/metrics, AI read-only tools, end-to-end test pass.

**M5 — Hardening & demo:** Owners: All members. Security and authorization tests, clean-clone setup, final diagrams/docs, test report, contribution sheet, demo rehearsal.

## 6. Cross-workflow testing ownership

This matrix is a minimum coverage guide. Add test IDs and results to the final test report. Testing is shared: the named owner leads, but connected module owners help diagnose and fix failures.

| Area | Lead(s) | Minimum scenarios |
|---|---|---|
| Identity and permissions | 1; all endpoint owners | Valid/invalid login, expired token, role denial, cross-user access, cross-merchant access. |
| Booking and pricing | 2; 6 | Validation boundaries, price examples, idempotency, history and receipt access. |
| Dispatch and delivery | 3; 7 | Assignment conflicts, courier ownership, valid/invalid transitions, failed attempt and proof. |
| Merchant integration | 4; 8 | Credential revocation, tenant isolation, webhook configuration, signature/retry/DLQ. |
| Tracking and realtime | 5; 6 | Current status, privacy, SSE reconnect/fallback, unknown tracking number. |
| Async pipeline | 8; 10 | DB commit → outbox → broker → consumer; duplicate message; broker/consumer failure; replay. |
| Platform | 9; all owners | Clean build, Compose startup, K8s readiness, config/secret injection, service connectivity. |
| AI assistant | 10; 1/2/5 | Tool validation, authorization, timeout/provider failure, no invented parcel facts. |

# 7. Shared contracts to settle before parallel development

**Identity:** Role names, token/session mechanism, Google OIDC callback flow, auth middleware interface, current-user context.

**API:** Base paths and versioning (e.g. /api/v1), JSON naming, pagination, validation errors, error envelope, correlation/request ID.

**Database:** Canonical parcel/status/attempt/assignment entities, migration naming/order, transaction ownership, IDs and timestamps.

**Parcel lifecycle:** Allowed transitions, who can perform each transition, how failed attempts and redelivery work, audit requirements.

**Pricing:** Currency, units, configurable formula, rounding, whether a booking stores a price snapshot.

**Public tracking:** Exact public fields; no phone numbers, full addresses, private notes, proof images, or merchant-only metadata.

**Events:** Event names, envelope, schema versioning, outbox table/interface, idempotency key, retry/DLQ policy.

**Webhook security:** Signature scheme, timestamp tolerance, retry schedule, SSRF controls, allowed destinations, replay behavior.

**Infrastructure:** Ports, service names, environment variable names, health endpoints, gateway/Ingress split, local versus Kubernetes topology.

**AI:** Allowed tools, authentication forwarding, output restrictions, timeout behavior, provider fallback, whether MCP is required or optional.

# 8. Definition of done for every member

- The feature matches the agreed acceptance criteria and published contract.
- Code is formatted and builds successfully; no unexplained TODOs block the required workflow.
- Unit tests and relevant integration tests pass; failures and known limitations are recorded.
- Authorization, validation, privacy, and failure cases are tested—not only the happy path.
- Database migrations are reproducible and safe to apply in the documented order.
- Configuration is externalized; .env.example contains placeholders only; no real secrets are committed.
- The module README includes setup, environment variables, endpoints, test commands, dependencies, and a short demo path.
- The feature is integrated into the shared branch and reviewed by at least one teammate.

# 9. Risk controls and practical recommendations

- Do not let the platform owner become the only person who can deploy the system. Every service owner must understand their Dockerfile, health endpoint, and configuration.
- Do not make Member 10 the only person writing tests. Every member writes feature tests; Member 10 coordinates end-to-end testing and helps with integration failures.
- Member 8 has a relatively large workload. Freeze the event envelope and outbox interface early, keep notification providers behind adapters, and use a local fake receiver for the demo.
- Keep AI/MCP scope deliberately small until booking, tracking, dispatch, and the async integration path work end-to-end.
- The assignment bonus may expect database and broker workloads in Kubernetes. Confirm rubric interpretation before relying on managed external services.

- Use mocks only to unblock parallel work; replace them with real integration tests before the final demonstration.
- Keep one canonical definition of parcel state and status history. Tracking, AI, and frontend code must not invent their own conflicting state models.

## 10. Team sign-off checklist

- ■ All ten members have accepted their primary ownership area and know their backup/support person.
- ■ Core API versus Tracking API boundary is agreed.
- ■ Roles, parcel state machine, pricing formula/currency, and failed-delivery semantics are agreed.
- ■ API contracts and event envelope are published before dependent implementation begins.
- ■ Database migration ownership and review process are assigned.
- ■ Repository, branch/PR policy, CI checks, and issue tracker are ready.
- ■ Milestones and integration dates are visible to all members.
- ■ Each member has a demonstrable deliverable and a short explanation of their contribution.

End of document — maintain this plan as implementation decisions are finalized.